

Computational Sociology

Data structures and algorithms: From networks to
agent-based models

Dr. Thomas Davidson

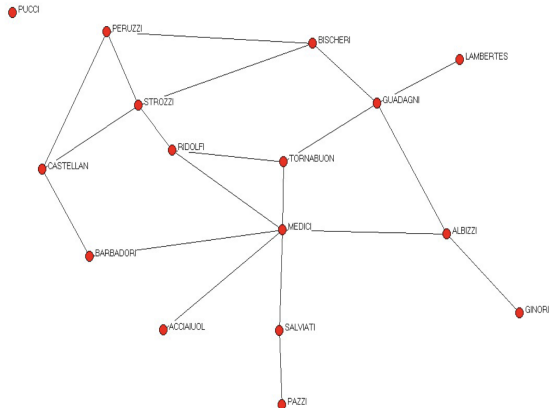
University of North Carolina at Chapel Hill

August 28, 2026

Plan

- ▶ Part I: Networks and agent-based models
- ▶ Part II: Data structures and programming
 - ▶ Basic types, vectors, matrices, lists
 - ▶ Logic, loops, and functions
- ▶ Part III: Putting it all together

Part I: Social networks and social network analysis



15th century Florentine marriage network (depicted in Jackson, Matthew O. 2010. *Social and Economic Networks*. Princeton University Press.)

Social networks

- ▶ A social network is a set of actors, or people, and the relationships between them
 - ▶ Family structures and kinship networks
 - ▶ Friendship and acquaintanceship networks
 - ▶ Organizations
 - ▶ Online social networks

Social networks

- ▶ Social network analysis studies these structures using mathematical and computational tools
 - ▶ Networks are represented using concepts from a branch of mathematics called graph theory
 - ▶ People or actors are represented as “nodes”
 - ▶ Relationships are represented as “edges”

Social network analysis

Visualizing networks

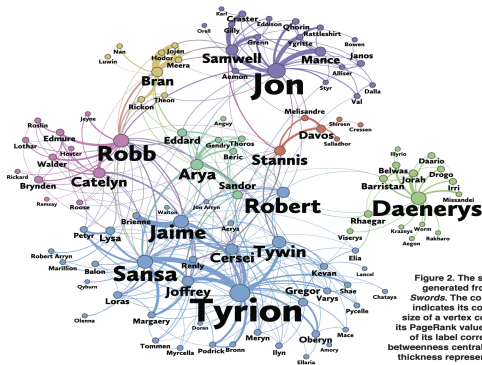


Figure 2. The social network generated from *A Storm of Swords*. The color of a vertex corresponds to its PageRank value, and the size of its label corresponds to its betweenness centrality. An edge's thickness represents its weight.

Characters mentioned in *Game of Thrones* books, annotated with structural information. From Beveridge, Andrew, and Jie Shan. 2016. "Network of Thrones." *Math Horizons* 23(4): 18–22.

Social network analysis

Big ideas: Homophily

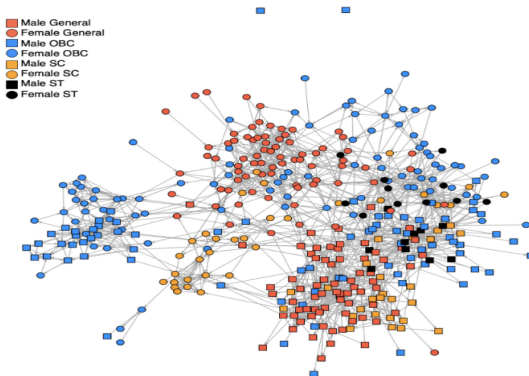


FIGURE 2 Network visualization of a selected village (Village #52).

Lee, Jaemin, and Mudit Kumar Singh. 2024. "Expansion, Cohesion and Diversity: The Network Advantages of Microfinance Groups in Indian Villages." *Journal of International Development* 36(1):559–86.

Social network analysis

Big ideas: The strength of weak ties

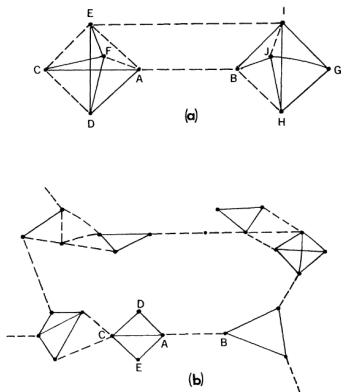


FIG. 2.—Local bridges. *a*, Degree 3; *b*, Degree 13. — = strong tie; --- = weak tie.

Granovetter 1973

Social network analysis

Big ideas: Structural holes

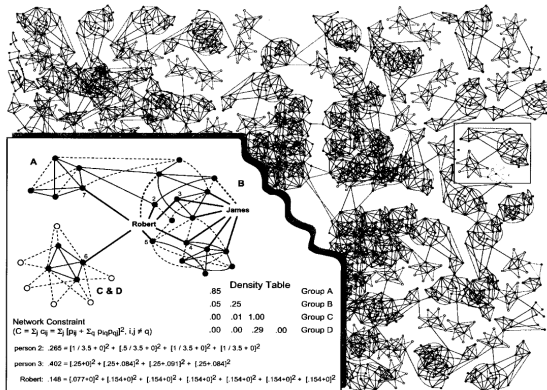
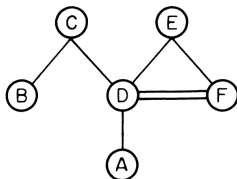


FIG. 1.—The small world of markets and organizations

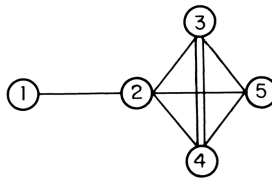
Burt 2004

Social network analysis

Big ideas: Duality of persons and groups



A-1. Interpersonal network

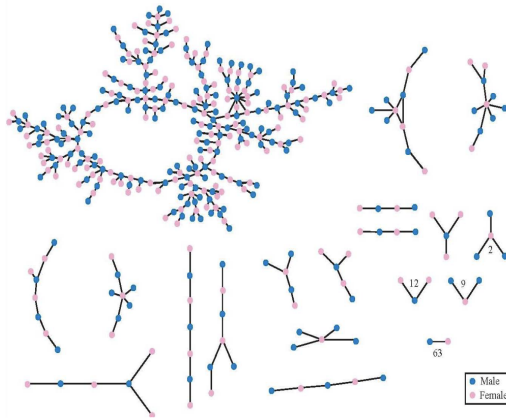


A-2. Intergroup network

Breiger 1974

Social network analysis

Spanning trees of sexual relationships in a high school



Bearman, Moody, and Stovel 2004

Social network analysis

Forbidden triads and triadic closure

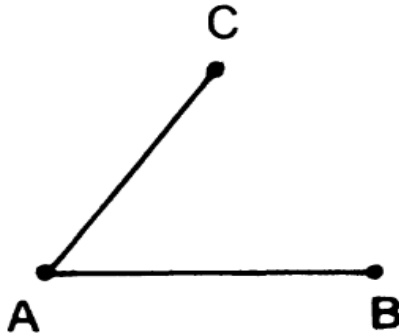


FIG. 1.—Forbidden triad

Granovetter 1973

Social network analysis

Micro-mechanism: Norms against cycles and “seconds”

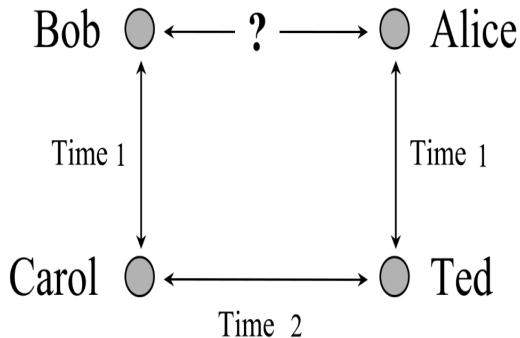
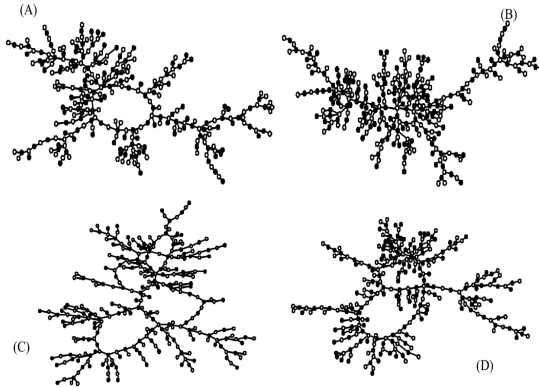


FIG. 8.—Hypothetical cycle of length 4

Social network analysis

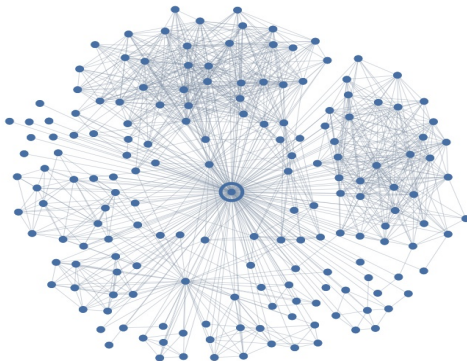
Simulated networks



Bearman, Moody, and Stovel 2004

Social network analysis

Can you find someone's partner from network structure alone?



Backstrom and Kleinberg 2014

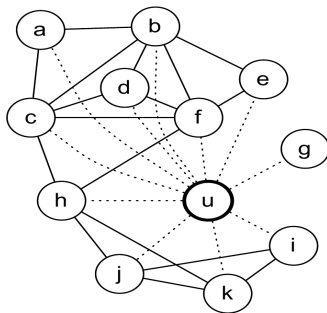
Social network analysis

Embeddedness and dispersion

- ▶ *Embeddedness* counts the friends two people share
- ▶ *Dispersion* measures whether mutual friends are themselves unconnected
 - ▶ The partner is usually the friend whose mutual connections reach across separate parts of the network

Social network analysis

Embeddedness and dispersion



Backstrom and Kleinberg 2014

Social network analysis

Embeddedness and dispersion

type	embed	rec.disp.	photo	prof.view.
all	0.247	0.506	0.415	0.301
married	0.321	0.607	0.449	0.210
married (fem)	0.296	0.551	0.391	0.202
married (male)	0.347	0.667	0.511	0.220
engaged	0.179	0.446	0.442	0.391
engaged (fem)	0.171	0.399	0.386	0.401
engaged (male)	0.185	0.490	0.495	0.381
relationship	0.132	0.344	0.347	0.441
relationship (fem)	0.139	0.316	0.290	0.467
relationship (male)	0.125	0.369	0.399	0.418

Figure 4. The performance of different measures for identifying spouses and romantic partners: the numbers in the table give the *precision at the first position* — the fraction of instances in which the user ranked first by the measure is in fact the true partner. Averaged over all instances, recursive dispersion performs approximately twice as well as the standard notion of embeddedness, and also better overall than measures based on profile viewing and presence in the same photo.

Backstrom and Kleinberg 2014

Social networks and computational social science

- ▶ *Custommade vs. readymade data:*
 - ▶ Add Health data is custommade
 - ▶ The Facebook data is readymade
 - ▶ The ubiquity of large-scale, dynamic network datasets creates new opportunities

Social networks and computational social science

- ▶ *Computational approaches to network analysis:*
 - ▶ Bearman et al. 2004 demonstrate how simulations can reveal micro-mechanisms underlying macro-structures
 - ▶ Large-scale network data facilitate analysis at unprecedented scale
 - ▶ Advances in computational power enable new types of network simulation (see Block, Stadtfeld, and Snijders 2019)

Introduction to agent-based modeling

From variables to actors

- ▶ Most quantitative social science is variable-centered
 - ▶ Regression forces us to think about the world in terms of what Andrew Abbott calls “general linear reality,” a social world composed of fixed entities with fixed attributes

Introduction to agent-based modeling

From variables to actors

- ▶ Agent-based modeling is the study of “social life as interactions among adaptive agents who influence one another in response to the influence they receive” (Macy and Willer 2002)
 - ▶ Rather than interactions between *variables*, ABMs consider interactions between *interdependent individuals*

Introduction to agent-based modeling

Emergence

- ▶ ABMs can reveal *emergent* properties of local interactions between agents and how they aggregate into system-level processes such as diffusion, polarization, and segregation
 - ▶ Complex, system-level patterns can develop without any centralized coordination
- ▶ Like historical sociology and ethnography, agent-based modeling is a *relational* approach, focusing on the context-dependent and contingent nature of social interaction

Introduction to agent-based modeling

Four key assumptions (Macy and Willer 2002)

- ▶ Agents are *autonomous*
 - ▶ There is no system-wide coordination
- ▶ Agents are *interdependent*
 - ▶ Agents respond to each other and to their environment
- ▶ Agents follow *simple rules*
 - ▶ Simple local rules can generate global complexity
- ▶ Agents are *adaptive* and *backward-looking*
 - ▶ Agents can alter their behavior through processes such as imitation and learning

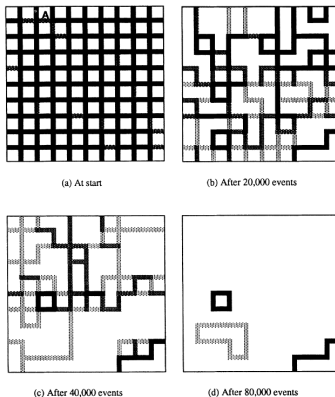
Introduction to agent-based modeling

Strengths of ABMs

- ▶ Virtual experiments to test causal mechanisms
 - ▶ Particularly useful where real-world experimentation is impractical
- ▶ Theory building and testing
 - ▶ Bridging between micro and macro levels of analysis
 - ▶ Varying the social structure *and* the agency of individuals

Modeling polarization

Local convergence and global polarization



Axelrod, Robert. 1997. "The Dissemination of Culture: A Model with Local Convergence and Global Polarization." *Journal of Conflict Resolution* 41 (2): 203–26.

Modeling polarization

Politics and lifestyle choices: Why do liberals drink lattes?

High correlations between political attitudes and lifestyle choices

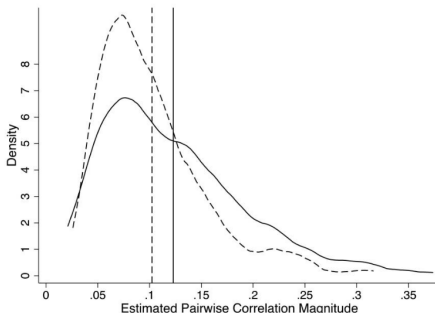
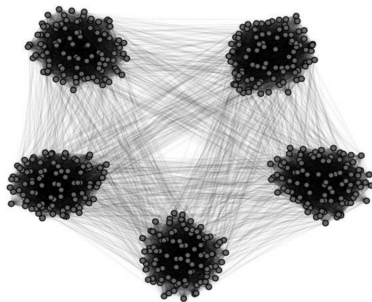


FIG. 2.—Magnitude of zero-order and partial correlation between GSS lifestyle items and ideological identity. Graphs plot Epanechnikov kernel density functions for both zero-order (solid lines) and partial (dashed lines) correlation magnitudes estimated from the mixed-effects model (see table A2 in the appendix). One value is plotted for each of the 216 item pairs. Time is set to 2010 to facilitate comparison across item pairs. The solid vertical reference line gives the mean predicted zero-order correlation magnitude across all item pairs in 2010 and the dashed vertical reference line gives the same value for partial correlation magnitude.

Modeling polarization

Politics and lifestyle choices: Why do liberals drink lattes?

- ▶ Mechanism: *network autocorrelation*
 - ▶ Feedback loops between spatial settings, relations, and lifestyle

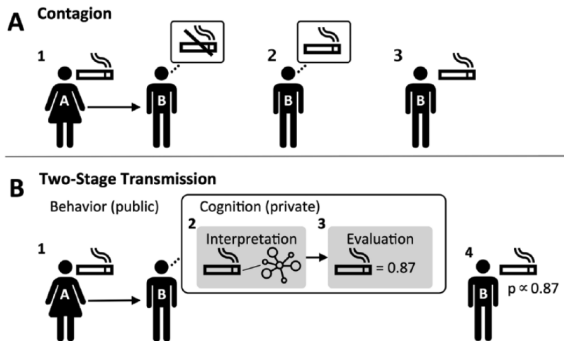


DellaPosta, Shi, and Macy 2015

Modeling polarization

Competing explanations: Associative diffusion

- Alternative mechanism: *belief structures* developed via observation, interpretation, and evaluation



Modeling polarization

Competing explanations: Associative diffusion

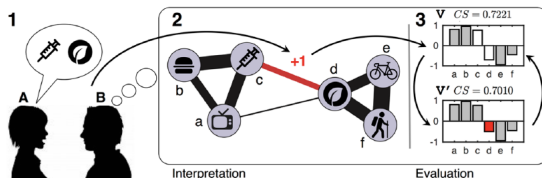


Figure 3. An Illustration of the Agent-Based Model Sequence

Note: (1) Agent B observes A express support for vaccinations and organic food (practices c and d); (2) B updates the corresponding element in his associative matrix, R (the edge connecting nodes c and d in the network representation of R); and (3) randomly updates his preference for organic food (practice d , resulting in preference vector V'), which is the weaker preference of the pair $\{c, d\}$ in his preference vector V . Because constraint satisfaction is reduced from .7221 to .7010, this preference update is rejected, and B 's preference vector V remains unchanged.

Goldberg and Stein 2018

Building agent-based models

Realism and external validity

- ▶ Bruch and Atwell (2015) distinguish between two types of realism in ABMs
 - ▶ *Low-dimensional realism*: simple, parsimonious models
 - ▶ *High-dimensional realism*: complex, complicated models
- ▶ Trade-offs:
 - ▶ The latter might be more realistic, but involve more parameters and may be less intelligible

Building agent-based models

Integrating real data to enhance realism

Network Externalities, Intergroup Inequality

TABLE 2
LINEAR REGRESSION OF ADOPTION LEVELS ON EXPERIMENTAL CONDITIONS

	ALL	RACE		INCOME		EDUCATION	
		Whites	Blacks	High	Low	BA	Less than High School
No network externalities	-.516**	-.536**	-.399**	-.685**	-.238**	-.611**	-.351**
General network externalities	.030**	.028**	.043**	.032**	.017**	.023**	.030**
Homophily = .25 ...	-.003**	-.001	-.012**	.009**	-.014**	.005**	-.011**
Homophily = .5 ...	-.005**	-.002**	-.024**	.017**	-.028**	.010**	-.024**
Homophily = .75 ...	-.011**	-.006**	-.040**	.024**	-.046**	.012**	-.043**
Homophily = 1	-.019**	-.012**	-.061**	.029**	-.067**	.015**	-.068**
Intercept	.618**	.647**	.454**	.925**	.249**	.788**	.392**
R ²	.99	.99	.97	.99	.96	.99	.96

NOTE.—All independent variables are binary. Both dependent and independent variables are measured on the final period of simulations ($t = 100$). Reference: homophily = 0; $N = 7,000$.

* $P < .05$.

** $P < .01$.

DiMaggio, Paul, and Filiz Garip. 2011. "How Network Externalities Can Exacerbate Intergroup Inequality." *American Journal of Sociology* 116 (6): 1887–1933.

Building agent-based models

High-dimensional realism using generative AI

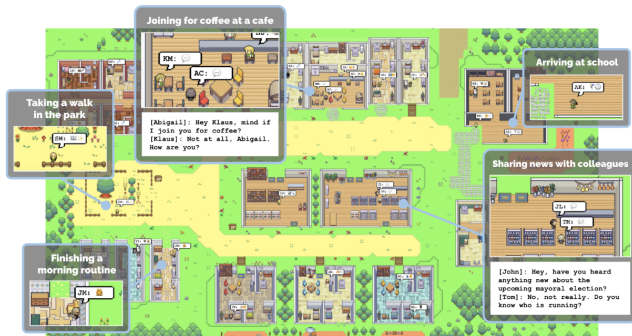


Figure 1: Generative agents are believable simulacra of human behavior for interactive applications. In this work, we demonstrate generative agents by populating a sandbox environment, reminiscent of The Sims, with twenty-five agents. Users can observe and intervene as agents plan their days, share news, form relationships, and coordinate group activities.

Park et al. 2023

Building agent-based models

Parameters and sensitivity

- ▶ Use theory to guide decisions regarding which parameters vary and which are fixed
 - ▶ Formalizations can be hard to operationalize
- ▶ Models can be extremely sensitive to small variations in parameters
 - ▶ Be careful to check for coding errors*
- ▶ Timing matters
 - ▶ Continuous-time vs. discrete-time
 - ▶ Asynchronous vs. synchronous updating

* See: van de Rijt, Arnout, David Siegel, and Michael Macy. 2009. "Neighborhood Chance and Neighborhood Change: A Comment on Bruch and Mare." *American Journal of Sociology* 114(4):1166–80 & Goldberg, Amir. 2021. "Associative Diffusion and the Pitfalls of Structural Reductionism." *American Sociological Review* 86(6):1205–10.

Schelling's segregation model

DYNAMIC MODELS OF SEGREGATION

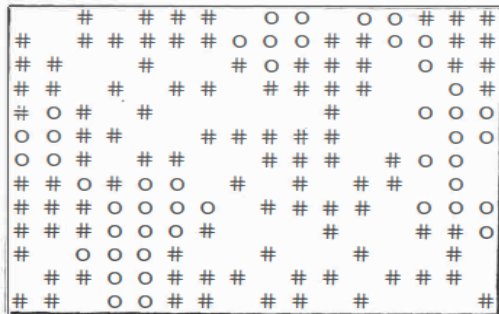


Fig. 13

Schelling, Thomas C. 1971. "Dynamic Models of Segregation." *Journal of Mathematical Sociology* 1: 143–86.

Schelling's segregation model

NetLogo

- ▶ Visit <https://www.netlogoweb.org>
- ▶ Click NetLogo Web
- ▶ Select "Sample Models/Social Science/Segregation" from dropdown

Part II: Data structures and programming

Building Schelling's model

- ▶ Agents belonging to two groups occupy cells on a grid
- ▶ Each agent looks at the neighbors surrounding it
- ▶ An agent with too few similar neighbors moves to an empty cell
- ▶ Repeat until nobody moves

Opening the RMarkdown file

- ▶ Open class project in RStudio
- ▶ Click pull (downwards blue arrow) in Git tab
- ▶ Open `lecture2-data-structures-and-algorithms.Rmd`, located in `2026/course-materials/slides/`.
- ▶ Scroll down to this line.

Objects

- ▶ R is an object-oriented language
 - ▶ We create *objects* of different *classes* such as numbers, strings, and matrices
 - ▶ Objects have *attributes*, properties such as their contents and dimensions
 - ▶ Objects are associated with *methods* that allow us to manipulate them
 - ▶ e.g. If A is a numeric object equal to 1, its add method means $A + A$ returns 2

Basic types

Four basic types will cover almost everything in this class. Here they store the attributes of a Schelling agent. R uses the `<-` operator for assignment.

```
# Integer ("int" for short)
group <- 1L           # which group the agent belongs to
# Numeric
threshold <- 0.5      # share of similar neighbors it requires
# Logical
happy <- FALSE        # is it content where it is?
# Character (also known as "strings")
label <- "blue"       # a name for the group
```

The other two are called `complex` and `raw`. See documentation

Basic types

Three commands for inspecting an object.

```
print(threshold) # Prints value in console
```

```
## [1] 0.5
```

```
class(threshold) # Shows class of object
```

```
## [1] "numeric"
```

```
typeof(group) # Shows type, not always equal to class
```

```
## [1] "integer"
```

Basic types

Comparison operators return a logical value, TRUE or FALSE.

```
group == 1L      # equals
```

```
## [1] TRUE
```

```
group != 2L      # is not
```

```
## [1] TRUE
```

```
0.25 >= threshold # is greater than or equal to
```

```
## [1] FALSE
```

Vectors

A vector is a collection of elements of the *same* type. The `c()` function *combines* values into a vector. Here are an agent's neighbors.

```
nbrs <- c(1, 2, 1, 0, 1, 2, 0, 2) # e.g. an agent's neighbors  
length(nbrs)
```

```
## [1] 8
```

Vectors

Square brackets, `[]`, are used for *indexing* specific cells.

```
nbrs[1] # First neighbor
```

```
## [1] 1
```

```
nbrs[8] # Last neighbor
```

```
## [1] 2
```

Vectors

An empty vector of N elements takes a default value depending on its type. These are useful for storing results.

```
N <- 5  
numeric(N)
```

```
## [1] 0 0 0 0 0
```

```
logical(N)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE
```

```
character(N)
```

```
## [1] "" "" "" "" ""
```

Vectors

What happens when different types are combined?

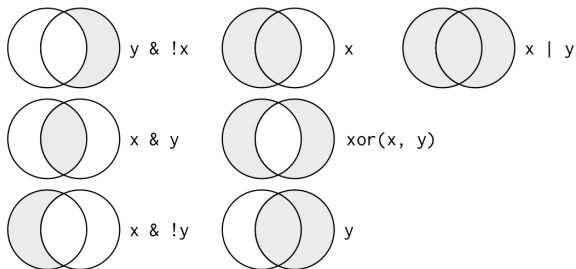
```
c(1, 2, "empty")
```

```
## [1] "1"      "2"      "empty"
```

```
typeof(c(1, 2, "empty"))
```

```
## [1] "character"
```

Boolean logic



Boolean logic

```
TRUE & FALSE    # and
```

```
## [1] FALSE
```

```
TRUE | FALSE    # or
```

```
## [1] TRUE
```

```
!TRUE           # not
```

```
## [1] FALSE
```

```
TRUE & !FALSE
```

```
## [1] TRUE
```

See the documentation for more on logic in R.

Boolean logic

Comparing a vector to a value returns a logical vector, one element at a time. The model needs two of these.

```
occupied <- nbrs != 0      # which positions hold an agent  
occupied
```

```
## [1]  TRUE  TRUE  TRUE FALSE  TRUE  TRUE FALSE  TRUE
```

```
same <- nbrs == group      # which hold an agent from the same group  
same
```

```
## [1]  TRUE FALSE  TRUE FALSE  TRUE FALSE FALSE FALSE
```

Boolean logic

`sum()` treats TRUE as 1 and FALSE as 0.

```
sum(same)                # N similar neighbors
```

```
## [1] 3
```

```
sum(occupied)           # N neighbors
```

```
## [1] 6
```

```
sum(same) / sum(occupied) # share similar
```

```
## [1] 0.5
```

Exercise: Vectors

An agent from group 2 has the neighborhood below. Is it happy?

```
group2 <- 2                # the agent's group  
nbrs2 <- c(1, 1, 2, 0, 1, 1, 0, 2) # its neighborhood
```

Randomness

Seeds ensure reproducibility. `sample` draws values uniformly.

```
set.seed(950)
sample(c(1, 2), 1) # assign an agent to a group
```

```
## [1] 1
```

```
sample(1:5, 1) # choose a row at random
```

```
## [1] 3
```

Use the help command, e.g. `?sample`, to find the arguments a function takes.

Matrices

A matrix is a two-dimensional structure that, like a vector, holds a single type. This one is the grid the agents live on.

```
n_rows <- 5
n_cols <- 5
grid <- matrix(0, nrow = n_rows, ncol = n_cols) # matrix of 0s
dim(grid)    # N rows, N columns
```

```
## [1] 5 5
```

```
grid
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    0    0    0    0    0
## [2,]    0    0    0    0    0
## [3,]    0    0    0    0    0
## [4,]    0    0    0    0    0
## [5,]    0    0    0    0    0
```

Matrices

Two-dimensional indexing to assign. Used here to assign positions or print the status.

```
grid[3, 3] <- 1    # an agent from group 1
grid[2, 2] <- 1
grid[3, 4] <- 2    # an agent from group 2
grid[4, 3] <- 2
grid[3, ]          # row 3
```

```
## [1] 0 0 1 2 0
```

```
grid[, 3]          # column 3
```

```
## [1] 0 0 1 2 0
```

Matrices

A range in place of a single index returns a submatrix. This represents an agent's Moore neighborhood.

```
i <- 3
j <- 3
nbrs <- grid[(i - 1):(i + 1), (j - 1):(j + 1)]
nbrs
```

```
##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    1    2
## [3,]    0    2    0
```

https://en.wikipedia.org/wiki/Moore_neighborhood

Matrices

The same comparisons work on the submatrix. We subtract one from each count to exclude self.

```
sum(nbrs == grid[i, j]) - 1  # similar neighbors
```

```
## [1] 1
```

```
sum(nbrs != 0) - 1          # neighbors of any kind
```

```
## [1] 3
```


Lists

A list holds elements of different types. Named arguments make its contents easy to retrieve with the \$ operator.

```
params <- list(n_rows = 5, n_cols = 5, threshold = 0.5)
params$threshold
```

```
## [1] 0.5
```

Lists

Single brackets return a sublist; double brackets its contents.

```
params[1]           # a list of length one
```

```
## $n_rows  
## [1] 5
```

```
params[[1]]        # the value inside it
```

```
## [1] 5
```

```
class(params[1])
```

```
## [1] "list"
```

```
class(params[[1]])
```

```
## [1] "numeric"
```

Lists

A list can represent the whole state of the model in a single object.

```
model <- list(grid = grid, params = params)
model$params$threshold
```

```
## [1] 0.5
```

Exercise: Lists

Combine \$ and square bracket indexing to return the value of the cell at row 3, column 4 of the grid stored in `model`.

If-else statements

- ▶ We often want to make a choice contingent on some piece of information
- ▶ If-else statements chain together one or more conditional actions
 - ▶ In Schelling's model, if an agent's neighborhood is not similar enough it moves; else it stays

If-else statements

The `if` is followed by a condition in parentheses. If the condition is met, the code inside the braces runs. If not, the code after `else` runs instead.

```
happiness <- 0.25

if (happiness < threshold) {
  print("move")
} else {
  print("stay")
}
```

```
## [1] "move"
```

Note the syntax. Putting `else` on the line below the closing brace throws an error.

If-else statements

Operations can be chained together using `else if`. This is used during the grid sweep.

```
if (grid[i, j] == 0) {  
    print("the cell is empty")  
} else if (happiness < threshold) {  
    print("move")  
} else {  
    print("stay")  
}
```

```
## [1] "move"
```

Loops

- ▶ Loops repeat an operation many times
 - ▶ *For-loops* have a defined start and end
 - ▶ *While-loops* iterate as long as a condition holds
- ▶ Schelling's model needs both
 - ▶ For-loop to visit every cell
 - ▶ While-loop to continue until nobody moves

For-loops

Here a loop assigns each of five agents to a random group and stores the result.

```
groups <- numeric(5)

for (k in 1:5) {
  groups[k] <- sample(c(1, 2), 1)
}

groups
```

```
## [1] 1 2 1 2 1
```

For-loops

```
import random

groups = []

for k in range(5):
    groups.append(random.choice([1, 2]))
```

The syntax varies across languages but the structure is the same, as this Python example shows.

For-loops

Loops can be *nested*. In this case, cycling through each row and column.

```
for (i in 1:n_rows) {  
  for (j in 1:n_cols) {  
    if (grid[i, j] != 0) {  
      print(paste("agent at", i, j))  
    }  
  }  
}
```

```
## [1] "agent at 2 2"  
## [1] "agent at 3 3"  
## [1] "agent at 3 4"  
## [1] "agent at 4 3"
```

While-loops

A while-loop runs while a condition is true and stops when it becomes false. *The condition should eventually be false to avoid an infinite loop.*

```
agents <- 0

while (agents < 3) {
  agents <- agents + 1
  print(agents)
}
```

```
## [1] 1
## [1] 2
## [1] 3
```

While-loops

This can be used to allow the simulation to continue indefinitely until all agents are happy, i.e. none moved.

```
all_happy <- FALSE

while (!all_happy) {
  moves <- 0
  # sweep the grid, moving unhappy agents
  # and adding one to moves for each
  if (moves == 0) {
    all_happy <- TRUE
  }
}
```

Note: There may be parameterizations where this results in an infinite loop. When might this occur?

Functions

- ▶ A function takes an input, performs a sequence of operations, and yields an output
- ▶ Functions make code modular and extendable
- ▶ Many functions are built into R and available in packages, but it is often useful to write our own

“You should consider writing a function whenever you’ve copied and pasted a block of code more than twice” *R4DS*

Functions

```
calculate_happiness <- function(i, j) { # input
  nbrs <- grid[max(i - 1, 1):min(i + 1, n_rows), # operations
               max(j - 1, 1):min(j + 1, n_cols)]
  similar <- sum(nbrs == grid[i, j]) - 1
  occupied <- sum(nbrs != 0) - 1
  return(similar / occupied) # output
}
```

```
calculate_happiness(3, 3)
```

```
## [1] 0.3333333
```

max and min keep the slice inside the grid to ensure a valid result when the agent sits on an edge.

Functions

Testing

- ▶ It is important to test a function rather than assume it works
 - ▶ Check that it handles valid inputs correctly
 - ▶ Check the *edge cases*, the inputs that require special handling
- ▶ Errors that produce no warning are harder to find than the ones that stop the code

Functions

Testing: agents on the edge of the grid

An agent in the corner has three neighboring positions rather than eight.

```
grid[1, 1] <- 1  
calculate_happiness(1, 1)
```

```
## [1] 1
```

The result is correct, but it rests on a single neighbor.

Functions

Testing: agents with no neighbors

The agent has no neighbors. What causes the error?

```
grid[5, 5] <- 1
happiness <- calculate_happiness(5, 5)
happiness
```

```
## [1] NaN
```

```
if (happiness < threshold) {print("move")}
```

```
## Error in 'if (happiness < threshold) ...':  
## ! missing value where TRUE/FALSE needed
```

Functions

Updating to handle the isolated agents

Note the condition and the two return statements.

```
calculate_happiness <- function(i, j) {  
  nbrs <- grid[max(i - 1, 1):min(i + 1, n_rows),  
               max(j - 1, 1):min(j + 1, n_cols)]  
  occupied <- sum(nbrs != 0) - 1  
  if (occupied == 0) {  
    return(1) # an isolated agent has no reason to move  
  }  
  similar <- sum(nbrs == grid[i, j]) - 1  
  return(similar / occupied)  
}  
calculate_happiness(5, 5)
```

```
## [1] 1
```

Style

A note on style

- ▶ Programming languages require a specific syntax to function, but they also have stylistic conventions
- ▶ The `styler` and `lintr` packages will style code automatically
- ▶ See <https://style.tidyverse.org/> for more on R style

Style

Recommendations

- ▶ Use short, informative variable names
- ▶ Adopt a consistent convention, either snake_case or CamelCase
- ▶ Use spacing to make code readable, `a <- 1` is preferable to `a<-1`
- ▶ Make complex operations modular
- ▶ Comment liberally for your future self and others

Part III: Putting it all together

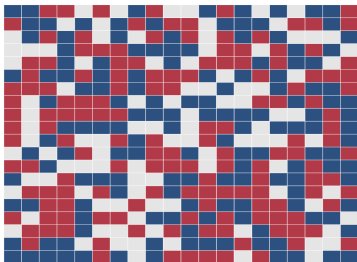
Schelling's segregation model in R

- ▶ Open `schelling.R`, located inside the code directory, and inspect the components

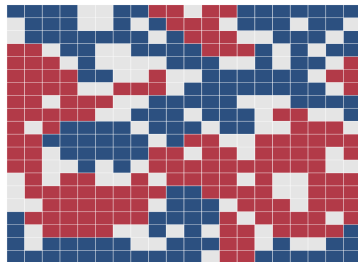
Schelling's segregation model in R

Before and after

Initial grid



Final grid



A threshold of 0.5 means an agent moves only when it is outnumbered where it lives. Preferences this mild still produce a sharply segregated grid.

Schelling's segregation model in R

Exercises

- ▶ Lower threshold to 0.3. What happens to the results?
- ▶ Raise it to 0.7. Does the model still settle?
- ▶ Change `n_agents`. What happens when the grid is denser?
- ▶ `calculate_happiness` divides by the number of occupied positions. Change it to divide by all neighboring positions. What happens?

Next week

- ▶ Substantive topic
 - ▶ Collecting digital trace data
- ▶ Technical topics
 - ▶ Application programming interfaces and web scraping
 - ▶ Working with tabular data